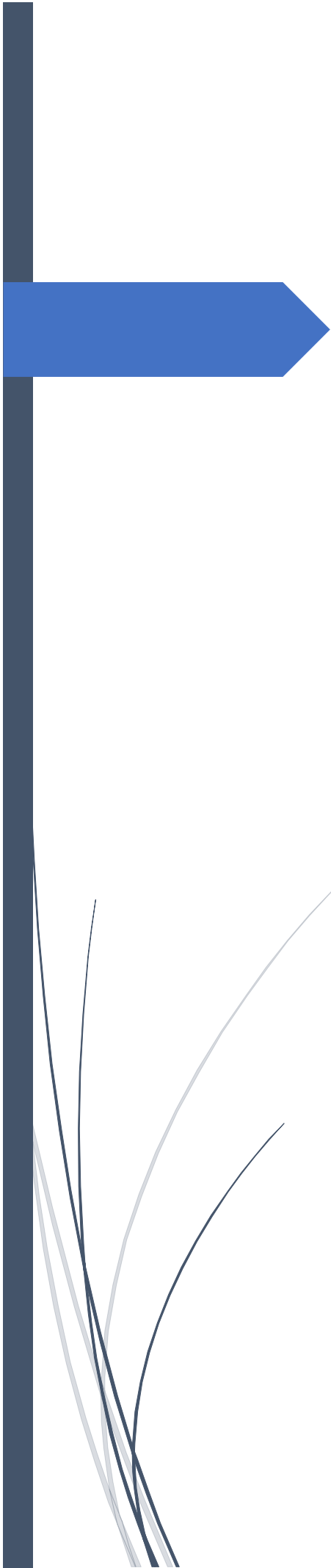




Report Ingestion & Verification Service

CS366 Service Proposal

Written by Akawat Moradsatian

6609681231

สารบัญ

ภาพรวมของบริการ (Service Overview).....	1
1. Service Owner	1
2. Service Purpose.....	1
3. Pain Point ที่แก้ไข	1
4. Target User.....	1
5. Service Boundary	2
6. Autonomy / Decision Logic	3
7. Owned Data.....	4
8. Linked Data (Reference Only)	5
9. Non-Functional Requirements	6
10. ลิงค์ที่เกี่ยวข้อง	6
Synchronous Function Contract.....	7
API Contract #1: Submit Raw Report (Ingest)	8
API Contract #2: List Pending Reports.....	11
API Contract #3: Verify Report (Decision)	14
API Contract #4: Get Report Detail (Insight Data).....	17
API Contract #5: Get Dashboard Stats (Summary).....	20
API Contract #6: Soft Delete / Archive.....	23
API Contract #7: Health Check	25
Asynchronous Function Contract.....	28
Message Contract #1: Report Verified Event	28
Message Contract #2: Report Status Change Event	33
Service Data.....	36

1. Reports Data (Owned by this service)	36
2. Report Audit Logs (Owned by this service)	38
3. StatsCounter Table (Atomic Counter)	39
4. ข้อมูลเพิ่มเติม	39
Service Architecture	41
1. ภาพแสดงการทำงานของแต่ละ Function.....	41
2. Components	41
3. Explanation เพิ่มเติม.....	42
Service Interaction.....	44
1. ภาพ Diagram.....	44
2. Upstream Services	44
3. Downstream Services.....	45
4. เหตุผลที่ออกแบบแบบนี้ (Technical Rationale)	46
Dependency Mapping.....	47
1. Incident Tracking Service.....	47
2. Amazon SQS (report.ingestion.queue.v1)	47
3. Amazon EventBridge (disaster.event.bus.v1).....	48
4. Amazon DynamoDB (Reports Table)	48
5. Google Gemini API	48

ภาพรวมของบริการ (Service Overview)

Report Ingestion & Verification Service

1. Service Owner

- Akawat Moradsatian (6609681231)

2. Service Purpose

- บริการนี้ทำหน้าที่เป็น Central Gateway หลักในการรับข้อมูลเหตุภัยพิบัติจากแหล่งข้อมูลภายนอกที่หลากหลาย ไม่ว่าจะเป็น Social Media Scrapers, Mobile Apps หรือ Third-party APIs ผ่านมาตรฐาน API โดยมีกระบวนการจัดการข้อมูลอย่างเป็นระบบ เริ่มต้นจากการใช้ Google Gemini API เพื่อทำการจำแนกกลุ่มข้อความเบื้องต้นให้เป็นหมวดหมู่ที่ชัดเจน
- จากนั้นระบบจะดำเนินการทำความสะอาดข้อมูลและลดความซ้ำซ้อนของรายงานที่ได้รับ และเข้าสู่ขั้นตอนสำคัญคือการประเมินความน่าเชื่อถือ โดยเลือกใช้พลังของ Google Gemini API (gemini-2.5-flash หรือ gemini-2.5-flash-lite หรือ gemini-2.0-flash) ในการทำ Trust Scoring เพื่อวิเคราะห์และให้น้ำหนักความน่าจะเป็นของเหตุการณ์จริงอย่างแม่นยำ กระบวนการทั้งหมดนี้จะเปลี่ยนจากข้อมูลดิบที่กระจัดกระจายให้กลายเป็นข้อมูลที่ผ่านการกรองและประเมินแล้วก่อนที่จะส่งต่อไปให้ Incident Service เพื่อนำไปสร้างเป็น Incident สำหรับการบริหารจัดการภัยพิบัติต่อไป

3. Pain Point ที่แก้ไข

- ในสภาวะวิกฤต ปริมาณข้อมูลที่หลั่งไหลเข้ามามีจำนวนมหาศาล (High Throughput) ซึ่งเกินขีดความสามารถและระยะเวลาที่บุคลากรจะสามารถประมวลผลหรือตรวจสอบได้ทันทั่วทั้งที่
- ข้อมูลรายงานกว่าร้อยละ 80 มักปรากฏลักษณะความซ้ำซ้อน (Duplicate) การให้ข้อมูลเท็จ (Fake News) หรือเป็นข้อความขยะ (Spam)
- ข้อมูลที่ได้รับจากแหล่งที่มาอันหลากหลาย ขาดความเป็นเอกภาพทางโครงสร้าง (Unstructured) ส่งผลให้เกิดความยุ่งยากในการนำไปประมวลผลหรือใช้งานต่อได้ทันที

4. Target User

- เจ้าหน้าที่ปฏิบัติการผู้รับผิดชอบด้านการตรวจสอบและยืนยันความถูกต้องของข้อมูล (เป็นผู้อนุมัติขั้นเด็ดขาดในขั้นตอน Verification)

- ระบบปฏิบัติการอัตโนมัติที่มีหน้าที่นำส่งข้อมูลเข้าสู่ API ของบริการนี้

5. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)
 - จัดการระบบ RESTful API เพื่อรองรับการนำเข้าข้อมูลดิบแบบ Raw JSON Payload จากแหล่งข้อมูลภายนอกทุกรูปแบบอย่างมีประสิทธิภาพ
 - ดำเนินการประเมินคะแนนความน่าเชื่อถือแบบ Trust Scoring โดยใช้ Google Gemini API และวิเคราะห์ระดับความรุนแรงของสถานการณ์ในเบื้องต้น เพื่อช่วยคัดกรองความสำคัญของเหตุการณ์
 - ตรวจสอบและคัดกรองข้อมูลซ้ำซ้อน (Deduplication & Idempotency) โดยใช้ External ID Check ผ่าน Global Secondary Index (gsi_source_external_id) เพื่อทำ Idempotency ป้องกันการรับข้อมูลซ้ำจากแหล่งที่มาเดียวกัน รวมทั้งประยุกต์ใช้ Haversine Formula ในการคำนวณระยะทางจากพิกัด GPS เพื่อทำ Geospatial Clustering โดยตรวจสอบเหตุการณ์ที่เกิดขึ้นในรัศมีไม่เกิน 200 เมตร และตรวจสอบภายในกรอบเวลา (Time Window) ที่ใกล้เคียงกันไม่เกิน 15 นาที และวิเคราะห์ความคล้ายคลึงของเนื้อหา (Content Similarity) เพื่อควรรวมรายงานที่มาจากเหตุการณ์เดียวกัน
 - พัฒนาส่วนติดต่อผู้ใช้งาน เพื่ออำนวยความสะดวกให้เจ้าหน้าที่ในการตรวจสอบ และใช้กลไกควบคุมสถานะเพื่อทำการอนุมัติ (Approve), ปฏิเสธ (Reject) หรือควรรวม (Merge) รายงานให้เป็นระเบียบ
 - ถ่ายทอดข้อมูลเหตุการณ์ ส่งต่อข้อมูลในรูปแบบ Asynchronous หลังจากยืนยันรายงานแล้ว เพื่อให้บริการอื่น ๆ ในระบบสามารถนำข้อมูลไปใช้งานต่อได้ทันที
- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
 - ไม่ครอบคลุมการพัฒนากระบวนการอัตโนมัติ เช่น Bot หรือ Scapper เพื่อดึงข้อมูลจากเว็บไซต์ภายนอก ซึ่งถือเป็นความรับผิดชอบของ External Client ในการนำส่งข้อมูลผ่าน API ด้วยตนเอง
 - ไม่ครอบคลุมการติดตามวงจรชีวิตและสถานะการดำเนินการกู้ภัย อาทิ ตำแหน่งปัจจุบันของยานพาหนะ หรือสถานะการปิดเหตุการณ์ ซึ่งเป็นความรับผิดชอบของ Incident Service

- ไม่ครอบคลุมการบริหารจัดการและสิ่งการทรัพยากรหรือยานพาหนะกู้ภัย ซึ่งเป็นความรับผิดชอบของ Dispatch Service

6. Autonomy / Decision Logic

- บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ
 - ในการคัดแยกสถานะรายงานอัตโนมัติ ระบบสามารถประเมินและเปลี่ยนสถานะของรายงานได้ทันทีตามเงื่อนไขที่กำหนด เพื่อลดภาระในการคัดกรองข้อมูลดิบ
 - หากระดับความน่าเชื่อถือ (Trust Score) ต่ำกว่า 30% ระบบจะปรับสถานะเป็น SPAM โดยอัตโนมัติ เพื่อตัดข้อมูลที่ไม่เป็นประโยชน์ออกจากระบบ
 - หากระดับความน่าเชื่อถือ (Trust Score) ตั้งแต่ 30% ขึ้นไป และตรวจสอบพบว่าข้อมูลตรงกับเงื่อนไขพื้นที่และเวลา (Geo/Time Match) ระบบจะปรับสถานะเป็น DUPLICATE เพื่อทำการควบรวมรายงาน
 - หากรายงานมีระดับความน่าเชื่อถือผ่านเกณฑ์และไม่ซ้ำซ้อนกับข้อมูลเดิม ระบบจะกำหนดสถานะเป็น PENDING_REVIEW เพื่อรอการตรวจสอบจากเจ้าหน้าที่ในลำดับถัดไป
 - ในกระบวนการจัดกลุ่มข้อมูล ระบบมีศักยภาพในการตัดสินใจได้เองว่ารายงานฉบับใหม่ที่ได้รับ เป็นเหตุการณ์เดียวกับรายงานที่มีอยู่เดิมหรือไม่ และดำเนินการจัดกลุ่มข้อมูลเข้าด้วยกันตามตรรกะที่กำหนดไว้
 - ในการวิเคราะห์เนื้อหาวิจัยระดับความรุนแรง ระบบสามารถตรวจจับคำสำคัญเชิงวิกฤต (Critical Keywords) เช่น "ติดอยู่", "หายใจไม่ออก" เพื่อใช้ในการประเมินระดับความรุนแรง (Severity) ของเหตุการณ์ได้โดยอัตโนมัติ
- การตัดสินใจอิงจาก
 - เกณฑ์วัดระดับความน่าเชื่อถือตาม Trust Score Thresholds โดย Trust Score น้อยกว่า 30% จะถือว่าจัดเป็น SPAM แต่หาก Trust Score มากกว่าหรือเท่ากับ 30% จะนำเข้าสู่กระบวนการตรวจสอบความซ้ำซ้อนและรอการรีวิว (หมายเหตุเพิ่มเติม ระบบมีการตั้งค่า CONFIG TRUST_HIGH_PRIORITY=80 เพื่อระบุรายงานที่มีความน่าเชื่อถือสูงมาก อย่างไรก็ตาม ปัจจุบันฟังก์ชัน Priority Queue ยังอยู่ในแผนการพัฒนา และยังไม่ได้ถูกนำมาใช้จัดลำดับการแสดงผลจริงในเวอร์ชันนี้)

- หลักเกณฑ์เชิงพื้นที่และเวลา (Spatio-temporal Rules) ใช้การคำนวณระยะทางที่คลาดเคลื่อนไม่เกิน 200 เมตร และระยะเวลาที่ห่างกันไม่เกิน 15 นาที เพื่อตัดสินว่าเป็นเหตุการณ์เดียวกัน
 - ฐานข้อมูลคำสำคัญ (Keyword Database) ใช้ในการเทียบเคียงคำศัพท์เพื่อประเมินระดับความรุนแรง (Severity Assessment) ช่วยให้เจ้าหน้าที่ทราบถึงความเร่งด่วนของสถานการณ์ได้ทันที
- ทั้งนี้ บริการสามารถดำเนินการตัดสินใจได้โดยอัตโนมัติภายใต้เงื่อนไขทางธุรกิจ (Business Rules) ที่กำหนด
 - โดยปราศจากความจำเป็นในการพึ่งพาการอนุมัติจากบุคลากรในขั้นตอนการคัดกรอง (Filtering) การจัดกลุ่ม (Grouping) และการจัดลำดับ (Sorting) เพื่อช่วยลดภาระงาน (Workload) ของเจ้าหน้าที่ปฏิบัติการ
 - อย่างไรก็ตาม ระบบยังคงต้องอาศัยการอนุมัติจากเจ้าหน้าที่ (Human-in-the-loop) ในขั้นตอนสุดท้าย (Verification) เพื่อยืนยันความถูกต้องของข้อมูลอย่างเป็นทางการ ก่อนดำเนินการส่งออกข้อมูลเพื่อสร้างเป็น Incident ในระบบจริง

7. Owned Data

- ข้อมูลที่บริการนี้เป็นเจ้าของและจัดเก็บ (Data Schema) ประกอบด้วย
- Raw Disaster Reports Table (Main Storage)
 - report_id (PK): รหัสอ้างอิงข้อมูลของรายงานดิบ (UUID)
 - source_metadata: ข้อมูลอ้างอิงแหล่งที่มา ประกอบด้วย Platform, User ID, Original Link และ gsi_source_external_id (สำหรับทำ Idempotency ตรวจสอบข้อมูลซ้ำจากแหล่งเดิม)
 - content_payload: ชุดข้อมูลเชิงเนื้อหา ประกอบด้วย ข้อความ (Text), ลิงก์รูปภาพ (Image URLs) และพิกัด GPS
 - analysis_result: ผลลัพธ์จากการวิเคราะห์ด้วย AI และระบบ
 - trust_score: ค่าคะแนนความน่าเชื่อถือ
 - sentiment_score: ค่าคะแนนความรู้สึกของเนื้อหา
 - suggested_category (Enum): ประเภทเหตุการณ์ที่ AI แนะนำ (เช่น FIRE, FLOOD, ACCIDENT)

- ai_reasoning (String): เหตุผลประกอบจาก AI ที่อธิบายที่มาของ Trust Score
 - ai_analysis_failed (Boolean): Flag ระบุในกรณีที่กระบวนการวิเคราะห์ด้วย AI เกิดข้อผิดพลาด
- verification_status (Enum): สถานะการดำเนินงานในปัจจุบัน ประกอบด้วย 7 สถานะหลักคือ
 - RECEIVED: ข้อมูลถูกรับเข้าระบบเรียบร้อยแล้ว
 - PENDING_REVIEW: ข้อมูลผ่านการกรองเบื้องต้นและรอเจ้าหน้าที่ตรวจสอบ
 - VERIFIED: ข้อมูลได้รับการยืนยันว่าเป็นเหตุการณ์จริง
 - SPAM: ข้อมูลถูกตัดสินว่าเป็นสแปมโดยระบบอัตโนมัติ
 - REJECTED: ข้อมูลถูกปฏิเสธโดยดุลยพินิจของเจ้าหน้าที่ (Human Rejection)
 - DUPLICATE: ข้อมูลที่ซ้ำซ้อนกับรายงานอื่นที่มีอยู่แล้ว
 - DELETED: ข้อมูลที่ถูกลบแบบ Soft Delete
- Deduplication & Management Fields:
 - potential_duplicates (List[UUID]): รายการ ID ของรายงานอื่นที่ระบบประเมินว่าอาจเป็นเหตุการณ์เดียวกัน
 - updated_at (DateTime): วันเวลาที่มีการแก้ไขข้อมูลล่าสุด
 - deleted_by / deleted_reason: ชื่อผู้ดำเนินการลบและเหตุผลในการลบ (สำหรับสถานะ DELETED)
- StatsCounter Table (Atomic Counter) เพื่อหลีกเลี่ยงการ Scan ข้อมูลทั้งฐานข้อมูลซึ่งมีค่าใช้จ่ายสูงและล่าช้า ระบบจึงจัดเก็บสถิติแยกไว้เพื่อการเข้าถึงที่รวดเร็ว
 - stat_key (PK): รหัสอ้างอิงของสถิติแต่ละประเภท (String) เช่น "2026-03-03#total_received" หรือ "status#verified#count"
 - stat_value: จำนวนนับสะสม โดยใช้กลไก Atomic Increment เพื่อความแม่นยำในการนับพร้อมกันหลาย Transaction

8. Linked Data (Reference Only)

- incident_id (Foreign Key - Reference Only): จัดเก็บเฉพาะรหัสอ้างอิง (ID) เพื่อประโยชน์ในการระบุความเชื่อมโยงว่ารายงานฉบับดังกล่าว ได้รับการนำไปสร้างเป็นเหตุการณ์ (Incident)

หมายเลขใดใน Incident Service (จำกัดสิทธิ์เฉพาะการเรียกดูข้อมูล โดยไม่อนุญาตให้แก้ไขข้อมูลใด ๆ ซ้ำเข้าไปยัง Incident Service)

9. Non-Functional Requirements

- **High Scalability (มีความสำคัญสูงสุด):** ระบบต้องมีศักยภาพในการรองรับภาระงานที่มีการบันทึกข้อมูลปริมาณมหาศาล (Write-Heavy Workload) ในช่วงเวลาวิกฤตหรือเกิดภัยพิบัติได้อย่างมีประสิทธิภาพ (ตัวอย่างเช่น การรองรับคำสั่งระดับ 10,000 requests/sec ผ่านระบบ Serverless Queue)
- **Data Integrity:** ข้อมูลที่เข้าสู่ระบบ (Ingestion) จะต้องมีความสมบูรณ์และไม่มีการสูญหายโดยเด็ดขาด (Zero Data Loss) ข้อมูลดิบทั้งหมดจะต้องได้รับการบันทึกลงสู่ฐานข้อมูล (Database) อย่างครบถ้วน แม้ในกรณีที่ระบบการตรวจสอบขัดข้องก็ตาม
- **Asynchronous Processing:** กระบวนการประมวลผลข้อมูลที่ใช้ทรัพยากรสูง อาทิเช่น การประมวลผลด้วย AI หรือ Deduplication จะต้องดำเนินการในรูปแบบ Background Process เพื่อป้องกันมิให้เกิดความล่าช้า หรือ Latency แก่ API ฝั่งรับข้อมูลขาเข้า

10. ลิงค์ที่เกี่ยวข้อง

- ลิงค์ GitHub [สามารถคลิกที่ข้อความนี้](https://github.com/Akawatmor/CS366-Serverless-ReportIngestion-VerificationService) หรือคัดลอกข้อความต่อไปนี้แล้วเปิดในเว็บเบราว์เซอร์ได้เลย

<https://github.com/Akawatmor/CS366-Serverless-ReportIngestion-VerificationService>

- สำหรับลิงค์ Endpoint โปรดใช้ลิงค์ต่อไปนี้เพื่อเรียกใช้งาน Microservice ที่กำหนด

ingestverify-366-dev.akawatmor.com

สำหรับ Endpoint เพิ่มเติมสามารถดูได้ที่เอกสารในส่วนถัดไป

- ลิงค์ Endpoint ข้างต้นอาจมีการเปลี่ยนแปลงได้ในอนาคต โปรดเช็คใน GitHub สำหรับข้อมูลล่าสุด

Synchronous Function Contract

Base URL: `ingestverify-366-dev.akawatmor.com/api/v1/`

เพิ่มเติม:

- Base URL จะนำมาจากเมื่อสั่ง Terraform output แล้วนำ URL นั้นมาเพิ่ม Record CNAME โดเมนข้างต้น โดยจะใช้งาน API ได้หลัง `/api/` path เป็นต้นไป เพื่อให้ `/` (root domain) สำหรับเป็นเว็บหลัก แนะนำรายละเอียด และ `/dashboard` สำหรับเป็นเว็บให้บริการตัวอย่าง ทั้งนี้ path API Contract ต่อไปนี้จะใส่ในรูปแบบ `/xxx` พึงรู้เสมอว่าต้องใส่ `/api/v1` ไว้ต่อหน้าเสมอ
- URL ผ่าน API Gateway มีการติด Rate Limited Default ที่ 50 Request/Sec โปรดทราบหากติด Rate Limited จะต้องรอให้หมดช่วงติด Rate Limited ก่อนจะใช้งานได้
- ใน Body Request หรือ Response จะใช้ JSON เป็นหลัก และจะไม่มีคอมเมนต์ `//` ในเนื้อข้อมูลเนื่องด้วยข้อกำหนดของไฟล์ JSON ทางผมใส่คอมเมนต์เพื่อให้เข้าใจความหมายของ Key-Value ให้เข้าใจตามบริบทมากขึ้น

API Contract #1: Submit Raw Report (Ingest)

1. ข้อมูลทั่วไป

- **Name:** Ingest Raw Report
- **Method:** POST
- **Path:** /reports
- **Type:** Synchronous (Request/Response แบบ Fire and Forget)

2. คำอธิบาย

- เป็นช่องทางหลักสำหรับนำเข้าข้อมูลจากภายนอก โดยใช้ Lambda (ingest_handler) ทำหน้าที่ตรวจสอบความถูกต้องของ Payload (Validation) ในเบื้องต้น ก่อนจะส่งข้อมูลเข้าสู่ Amazon SQS เพื่อรอการประมวลผลในลำดับถัดไป ช่วยให้ระบบสามารถตอบกลับผู้ใช้งานได้ทันทีและรองรับปริมาณข้อมูลมหาศาลได้ดี

3. Request

- **Path/Query Parameters:**
 - (None)
- **Headers:**
 - **Content-Type:** application/json
 - **X-API-Key:** <Client-Key> (เพื่อระบุว่าเป็นข้อมูลจาก Partner เจ้าไหน เช่น TwitterScraper, OfficialApp)
- **Body:**
 - JSON Payload as below example:

```
{
  "reporter_source": "TWITTER",
  // ENUM: [TWITTER, FACEBOOK, LINE, OFFICIAL_APP,
  IOT_SENSOR]
  "reporter_id": "@user123",
  // User ID หรือ Phone Number ของผู้แจ้ง
  "raw_content": "Fire reported near Central World! #BKKFire",
  // ข้อความดิบจากการรายงาน
  "media_urls": [
```

```

// Array Urls สำหรับหลายรูป
    "https://img.host/fire123.jpg",
    "https://video.host/clip.mp4"

],
"geo_location": {
    "lat": 13.746123,
    "lon": 100.539123
},
"timestamp": "2026-02-18T14:30:00Z"
// เวลาที่เกิดเหตุ (สำคัญมากสำหรับการเรียงลำดับ)
}

```

4. Validation

- reporter_source ต้องอยู่ใน ENUM ที่กำหนด
- raw_content หรือ media_urls ต้องมีอย่างน้อย 1 อย่าง มิฉะนั้นอาจถูกปรับไปยัง SPAM ได้
- ขนาด Payload รวมต้องไม่เกิน Limit ของ API Gateway/SQS เช่น ไม่เกิน 256KB

5. Response

- Success: (202 Accepted) พร้อม report_id และ status

```

{
    "status": "QUEUED",
    "report_id": "r-550e8400-e29b", // UUID ที่สร้างขึ้นที่
    "message": "Report accepted and queued for processing.",
}

```

- Error:
 - 400 Bad Request: ข้อมูลไม่ครบ (เช่น ขาด raw_content หรือ reporter_source)
 - 401 Unauthorized: API Key ไม่ถูกต้อง
 - 429 Too Many Requests: กรณียิงรัวเกิน Rate Limit ที่กำหนด
 - 500 Internal Server Error: Queue ล่ม

```
{  
  "error": true,  
  "message": "Too Many Requests",  
  "detail": "Rate limit exceeded. Please try again later."  
}
```

6. Dependency

- ระบบพึ่งพา SQS ในการรับช่วงต่อข้อความเพื่อทำ Asynchronous Processing หาก SQS ไม่สามารถให้บริการได้ ระบบจะส่งกลับเป็นสถานะ 500 Internal Server Error

7. Reliability

- มีระดับความน่าเชื่อถือสูง เนื่องจากใช้ Lambda ทำหน้าที่เพียงแค่ Validate ข้อมูลตามกฎธุรกิจที่กำหนด แล้วเขียนลง SQS ทันที กระบวนการนี้ช่วยลด Latency และรับประกันว่าข้อมูลจะถูกจัดเก็บเข้าคิวเพื่อรอการประมวลผลต่ออย่างแน่นอนแม้ว่า Service ส่วนอื่นจะทำงานหนักอยู่ก็ตาม

API Contract #2: List Pending Reports

1. ข้อมูลทั่วไป

- **Name:** List Pending Reports
- **Method:** GET
- **Path:** /reports
- **Type:** Synchronous (Request/Response)

2. คำอธิบาย

- บริการสำหรับเจ้าหน้าที่เพื่อดึงรายการรายงานเหตุภัยพิบัติที่ผ่านการคัดกรองเบื้องต้นแล้ว (สถานะ PENDING_REVIEW) เพื่อนำมาพิจารณาตรวจสอบความถูกต้อง (Verify) หรือดำเนินการอื่น ๆ ต่อไป

3. Request

- **Path/Query Parameters:**
 - status=PENDING_REVIEW
 - min_trust_score=50
 - limit=20 (Optional. Default: 5, Max: 100)
 - last_evaluated_key (Optional. ใช้สำหรับ Cursor-based Pagination เพื่อดึงข้อมูลหน้าถัดไป)
- **Headers:**
 - X-API-Key: <Client-Key> (ใช้ในการยืนยันตัวตนและตรวจสอบสิทธิ์เข้าถึงผ่าน Usage Plan)
- **Body:**
 - (None)

4. Validation

- status ต้องเป็นค่าที่อนุญาต (เช่น PENDING_REVIEW, VERIFIED)
- limit ต้องเป็นตัวเลขบวกไม่เกิน 100

5. Response

- Success: (200 OK)

```
{
  "data": [
    {
      "report_id": "r-550e8400-e29b-41d4-a716-446655440000",
      "content": "Fire near Central...",
      "trust_score": 85,
      "suggested_category": "FIRE",
      "time_ago": "5 mins"
    }
  ],
  "total_count": 1,
  // หมายถึงจำนวนรายการในหน้าปัจจุบัน (Current Page Count)
  "next_token": "eyJJSZXBvcnRJZCI6IHsiUyI6ICJyLTU1MGU4NDA..."
  // last_evaluated_key สำหรับดึงหน้าถัดไป
}
```

- Error:

- 400 Bad Request: Query Params ผิดรูปแบบ
- 401 Unauthorized: Token หมดอายุหรือไม่ถูกต้อง
- 403 Forbidden: ไม่มีสิทธิ์เข้าถึง (Role ไม่ถึง)

```
{
  "error": true,
  "message": "Unauthorized",
  "detail": "Invalid or expired API Key."
}
```

6. Dependency

- Amazon DynamoDB: ระบบพึ่งพาการ Query ข้อมูลจากฐานข้อมูลโดยตรงผ่าน Global Secondary Index (GSI) หากฐานข้อมูลขัดข้องจะส่งกลับเป็นสถานะ 500

7. Reliability

- เพื่อให้การดึงข้อมูลมีประสิทธิภาพสูงและประหยัด Resource (RCUs) ระบบจึงใช้งานผ่าน GSI: `gsi_status_ingested` โดยมีรายละเอียดดังนี้คือ
 - Partition Key: `validation_status`
 - Sort Key: `ingested_at`
 - Projection: ALL (เพื่อให้ได้ข้อมูลครบถ้วนโดยไม่ต้องย้อนกลับไปอ่าน Table หลัก)
- รองรับการขยายตัวด้วยระบบ Pagination แบบ Cursor-based ช่วยให้การดึงข้อมูลปริมาณมากไม่ส่งผลกระทบต่อ Latency ของระบบ

API Contract #3: Verify Report (Decision)

1. ข้อมูลทั่วไป

- **Name:** Verify Report (Decision)
- **Method:** PATCH
- **Path:** /reports/{report_id}
- **Type:** Synchronous (Request/Response)

2. คำอธิบาย

- ใช้สำหรับให้เจ้าหน้าที่ หรือระบบอัตโนมัติ ยืนยันสถานะความถูกต้องของรายงาน หากสถานะถูกเปลี่ยนเป็น VERIFIED ระบบจะดำเนินการส่ง Event ไปยัง EventBridge เพื่อสร้าง Incident ต่อในรูปแบบ Asynchronous

3. Request

- **Path/Query Parameters:**
 - report_id (UUID): ไอดีของรายงานที่ต้องการจัดการ
- **Headers:**
 - Content-Type: application/json
 - X-API-Key: <Client-Key> (ยืนยันตัวตนและสิทธิ์ผ่าน Usage Plan)

4. Body:

- JSON Payload as below example:

```
{
  "validation_status": "VERIFIED",
  // ENUM: [VERIFIED, SPAM, DUPLICATE, REJECTED]
  // (หมายเหตุ: ไม่รองรับสถานะ NEEDS_MORE_INFO ในขณะนี้)

  "reviewer_id": "officer_007",
  // ID เจ้าหน้าที่ผู้ทำรายการ (Required ถ้าเป็น VERIFIED)
  "reviewer_notes": "Confirmed via CCTV feed.", // บันทึกเพิ่มเติม
  "link_to_incident_id": "inc-1234-5678"
  // OPTIONAL: ใส่เมื่อต้องการ Merge เข้า Incident เดิมที่มีอยู่แล้ว
  // หากเป็น null และ status=VERIFIED ระบบจะสร้าง Incident ใหม่
}
```

5. Validation

- State Transitions: ระบบจะอนุญาตให้เปลี่ยนสถานะตามข้อมูลด้านล่างเท่านั้น (รูปแบบ From -> To Status)
 - RECEIVED -> PENDING_REVIEW, SPAM, REJECTED, DUPLICATE
 - PENDING_REVIEW -> VERIFIED, SPAM, REJECTED, DUPLICATE
 - VERIFIED-> (Terminal State) - ไม่สามารถเปลี่ยนได้ ยกเว้น DELETED
 - REJECTED / SPAM / DUPLICATE -> (Terminal State) - ไม่สามารถเปลี่ยนได้
- การ Optimistic Locking
 - เมื่ออัปเดต DynamoDB จะใช้
 - ConditionExpression:"validation_status = :expected_status"
 - เพื่อป้องกันการเขียนทับในกรณีที่ผู้อื่นจัดการรายงานนี้ไปก่อนแล้ว (หากไม่ผ่านจะคืนค่า 409 Conflict)
- ถ้า validation_status เป็น VERIFIED ต้องมี reviewer_id เสมอ

6. Response

- Success: (200 OK)

```
{
  "report_id": "r-550e8400-e29b-41d4-a716-446655440000",
  "validation_status": "VERIFIED",
  "action_taken": "TRIGGER_NEW_INCIDENT",
  // ค่าที่เป็นไปได้: TRIGGER_NEW_INCIDENT,
  MERGED_EXISTING_INCIDENT, NO_ACTION
  "updated_at": "2026-03-03T15:00:00Z"
}
```

7. Error:

- 400 Bad Request: ส่ง State ที่ไม่อนุญาต
- 404 Not Found: ไม่พบ report_id นี้
- 409 Conflict: รายงานนี้ถูกจัดการไปแล้วโดยผู้อื่น

```
{
  "error": true,
  "message": "Bad Request",
  "detail": "validation_status is invalid!"
}
```

ในงานพื้นหลัง เมื่อตอบกลับ 200 เรียบร้อยแล้ว ระบบจะทำการ Publish ReportVerifiedEvent ไปยัง Amazon EventBridge เพื่อให้ Incident Service ทำงานต่อทันที

8. Dependency

- Internal Dependency โดยพึ่งพาการเขียนข้อมูลลง DynamoDB และการส่ง Event เข้า EventBridge
- Asynchronous Flow ในการสร้าง Incident จริงจะถูกจัดการโดยบริการอื่นผ่านระบบ Event-Driven เพื่อลด Wait Time ของหน้าบ้าน

9. Reliability

- ระบบไม่ได้ใช้ DynamoDB Transactions เพื่อประสิทธิภาพสูงสุด แต่ใช้ลำดับการทำงานแบบ Sequential คือ
 - update_item: อัปเดตสถานะใน Table หลัก (พร้อม Conditional Check)
 - put_item: บันทึกประวัติการจัดการลงใน Audit Log Table
 - put_events: ส่งสัญญาณเหตุการณ์ไปยัง EventBridge
- หากขั้นตอนแรก (Update) ล้มเหลว ขั้นตอนอื่นจะไม่ทำงาน เพื่อรักษาความถูกต้องของข้อมูล (Data Integrity)

API Contract #4: Get Report Detail (Insight Data)

1. ข้อมูลทั่วไป

- **Name:** Get Report Detail (Insight Data)
- **Method:** GET
- **Path:** /reports/{report_id}
- **Type:** Synchronous (Request/Response)

2. คำอธิบาย

- ใช้สำหรับดึงข้อมูลรายละเอียดเชิงลึกของแต่ละรายงาน ประกอบด้วยข้อมูลจากผู้แจ้ง, เนื้อหา Multimedia, พิกัด GPS, ผลการวิเคราะห์จาก AI (เช่น Reasoning และ Tags) รวมถึงข้อมูลการยืนยันสถานะจากเจ้าหน้าที่เพื่อใช้ในการพิจารณาเหตุการณ์

3. Request

- **Path/Query Parameters:**
 - report_id (UUID): ไอดีของรายงานที่ต้องการดึงข้อมูล
- **Headers:**
 - X-API-Key: <Client-Key> (ยืนยันตัวตนและสิทธิ์การเข้าถึง)
- **Body:**
 - (None)

4. Validation

- report_id ต้องเป็นค่าที่ถูกต้องและไม่เป็นค่าว่าง

5. Response

- Success: (200 OK)

```
{
  "report_id": "r-550e8400-e29b-41d4-a716-446655440000",
  "reporter_info": {
    "source": "TWITTER",
    "reporter_id": "@somchai_123",
    "source_external_id": "tweet-123456789"
    //หมายเหตุ: ข้อมูลเชิงลึกของ Account เช่น followers_count
    ยังไม่รองรับในเวอร์ชันนี้
  },
}
```

```

"content": {
  "text": "ไฟไหม้ร้านทอง เยาวราช ตอนนี้เลย!",
  "images": ["https://s3.aws.../img1_hd.jpg"],
  "video": "https://s3.aws.../vid1.mp4",
  "geo_location": {
    "lat": 13.746123,
    "lon": 100.539123
  },
  "event_timestamp": "2026-03-03T10:00:00Z"
// เวลาที่เกิดเหตุจริง
},
"analysis": {
  "trust_score": 88,
  "ai_reasoning": "Detected fire and smoke in image. Multiple users
reporting same location.",
  "ai_analysis_tags": ["Fire", "Smoke", "High-Urgency"],
  "ai_analysis_failed": false,
  "suggested_category": "FIRE",
  "potential_duplicates": ["r-999", "r-888"]
},
"verification": {
  "status": "PENDING_REVIEW",
  "verified_by": "officer_007",
  "verification_notes": "Confirmed via CCTV feed.",
  "linked_incident_id": "inc-1234"
// ID Incident ที่เชื่อมต่อ (ถ้ามี)
},
"created_at": "2026-03-03T10:05:00Z",
"updated_at": "2026-03-03T10:10:00Z"
}

```

- Error:
 - 404 Not Found: ไม่พบข้อมูล

```
{  
  "error": true,  
  "message": "Not Found",  
  "detail": "Data from id not found!"  
}
```

6. Dependency

- Amazon DynamoDB: พึ่งพาการดึงข้อมูลจาก Table หลักเท่านั้น
- ไม่มี External Dependency (ไม่เรียก Service อื่น)

7. Reliability

- ระบบดึงข้อมูลโดยใช้ GetItem ของ DynamoDB ซึ่งเป็นการเข้าถึงข้อมูลผ่าน Partition Key โดยตรง ทำให้ได้รับข้อมูลที่รวดเร็วที่สุด (Low Latency) และมีความแม่นยำสูง (Strong Consistency)

API Contract #5: Get Dashboard Stats (Summary)

1. ข้อมูลทั่วไป

- **Name:** Get Dashboard Stats (Summary)
- **Method:** GET
- **Path:** /reports/stats
- **Type:** Synchronous (Request/Response)

2. คำอธิบาย

- ดึงตัวเลขสรุปภาพรวมสำหรับ Dashboard เพื่อให้ผู้บัญชาการหรือเจ้าหน้าที่ระดับสูงประเมินสถานการณ์ได้แบบ Real-time โดยเป็นการอ่านข้อมูลจากตารางสถิติที่สรุปไว้แล้ว (Pre-aggregated Stats) เพื่อประสิทธิภาพสูงสุด

3. Request

- **Path/Query Parameters:**
 - timeframe=today (Optional: today, last_24h, last_7d)
 - region=bkk (หมายเหตุ: มีการรับ Parameter นี้ไว้ แต่ปัจจุบันยังไม่ได้ Implement ระบบกรองตามพื้นที่จริง)
- **Headers:**
 - X-API-Key: <Client-Key> (ยืนยันตัวตนเจ้าหน้าที่)
- **Body:**
 - (None)

4. Validation

- timeframe ต้องเป็นค่าที่ระบบรองรับ (Allowed Values)

5. Response

- Success: (200 OK)

```
{
  "timestamp": "2026-03-03T12:00:00Z",
  "summary": {
    "total_received_today": 1500,
    "pending_review": 50,
    "verified_incidents": 120,
```

```

    "spam_rejected": 1330
  },
  "trending_keywords": [],
  "heatmap_data": []
  // [Planned] อยู่ระหว่างการพัฒนา (Future Enhancement)
}

```

- Error:

- 500 Internal Server Error: ระบบคำนวณสถิติขัดข้อง

```

{
  "error": true,
  "message": "Internal Server Error",
  "detail": "Cannot retrieve statistical data at this moment."
}

```

6. Dependency

- Amazon DynamoDB (StatsCounter Table) โดยพึ่งพาการอ่านข้อมูลจาก Table ที่ใช้เก็บ Counter โดยเฉพาะ
- ไม่มี External Dependency (ไม่เรียก Service อื่น)

7. Reliability

- เพื่อหลีกเลี่ยงการ Scan ตารางรายงานหลัก (Main Table) ซึ่งมีค่าใช้จ่ายสูงและส่งผลต่อ Latency ระบบจึงใช้กลไก Atomic Counter ในตาราง StatsCounter คือ
 - Key Pattern: ใช้รูปแบบ {date}#{stat_name} เช่น 2026-03-03#total_received เพื่อจัดเก็บข้อมูลรายวัน
 - Atomic Increment: ระบบประมวลผลหลังบ้าน (Worker) จะใช้คำสั่ง UpdateExpression: ADD stat_value :inc เพื่อเพิ่มจำนวนนับทุกครั้งที่มีการประมวลผลรายงานสำเร็จ ทำให้ข้อมูลแม่นยำแม้มีการส่งข้อมูลเข้ามาพร้อมกันจำนวนมาก

- Efficiency: เมื่อ API นี้ถูกเรียก ระบบจะใช้ BatchGetItem ดึงเฉพาะ Key ที่เกี่ยวข้อง มาแสดงผลทันที ทำให้ได้ความเร็วระดับ Milliseconds โดยไม่โหลดระบบฐานข้อมูลหลัก

API Contract #6: Soft Delete / Archive

1. ข้อมูลทั่วไป

- **Name:** Soft Delete / Archive
- **Method:** DELETE
- **Path:** /reports/{report_id}
- **Type:** Synchronous (Request/Response)

2. คำอธิบาย

- ใช้สำหรับปิดการมองเห็นของรายงาน (Soft Delete) โดยการเปลี่ยนสถานะเป็น DELETED เหมาะสำหรับการนิข้อมูลที่ใช้ทดสอบระบบ, ข้อมูลที่ละเมิดนโยบาย PDPA หรือข้อมูลที่เจ้าหน้าที่พิจารณาแล้วว่าควรนำออกจากระบบถาวร

3. Request

- **Path/Query Parameters:**
 - report_id (UUID): ไอดีของรายงานที่ต้องการลบ
- **Headers:**
 - X-API-Key: <Client-Key> (หมายเหตุ: ปัจจุบันระบบยังไม่ได้ Implement Role-based Access Control (RBAC) ดังนั้นทุก Client ที่มี API Key ที่ถูกต้องจะสามารถดำเนินการลบได้)
- **Body:**

```
{
  "reason": "Contains sensitive personally identifiable information (PII)",
  "deleted_by": "admin_001"
}
```

4. Validation

- reason (Required): ต้องระบุเหตุผลในการลบเสมอ เพื่อใช้ในการบันทึก Audit Log
- deleted_by (Required): ต้องระบุชื่อหรือไอดีผู้ดำเนินการลบเพื่อความโปร่งใส
- report_id ต้องมีอยู่จริงในระบบและต้องไม่ได้อยู่ในสถานะ DELETED อยู่ก่อนแล้ว

5. Response

- Success: (200 OK)

```
{
  "report_id": "r-550e8400-e29b-41d4-a716-446655440000",
  "status": "DELETED",
  "message": "Report has been archived and removed from public view."
}
```

- Error:

- 403 Forbidden: ไม่มีสิทธิ์ลบ
- 404 Not Found: ไม่พบ Report

```
{
  "error": true,
  "message": "Not Found",
  "detail": "Report not found or already deleted."
}
```

หมายเหตุ: ระบบจะคืนค่า 404 Not Found ในกรณีที่ไม่มี report_id หรือรายงานนั้นถูกเปลี่ยนสถานะเป็น DELETED ไปก่อนหน้านี้แล้ว

6. Dependency

- Amazon DynamoDB: พึ่งพาการอัปเดตสถานะใน Table หลัก
- ไม่มี External Dependency (ไม่เรียก Service อื่น)

7. Reliability

- ระบบดำเนินการแบบ Soft Delete โดยการอัปเดตฟิลด์ validation_status เป็น DELETED พร้อมทั้งบันทึก deleted_by และ reason ลงใน Table หลัก
- มีการบันทึกประวัติการลบลงใน Audit Logs Table อย่างละเอียด เพื่อให้สามารถตรวจสอบย้อนหลังได้ในกรณีที่เกิดข้อผิดพลาดหรือต้องการกู้คืนข้อมูล (Manual Recovery)

API Contract #7: Health Check

1. ข้อมูลทั่วไป

- **Name:** Health Check
- **Method:** GET
- **Path:** /health
- **Type:** Synchronous (Request/Response)

2. คำอธิบาย

- ทำหน้าที่ตรวจสอบสถานะความพร้อมใช้งาน (Readiness & Liveness) ของบริการและส่วนประกอบที่เกี่ยวข้องทั้งหมด เพื่อให้มั่นใจว่าระบบสามารถประมวลผลข้อมูลรายงานภัยพิบัติได้อย่างถูกต้อง

3. Request

- **Path/Query Parameters:**
 - None
- **Headers:**
 - (None) หมายเหตุ: Endpoint นี้เป็น Public ไม่ต้องการยืนยันตัวตน (No Auth) เพื่อให้ระบบ Monitoring ภายนอกสามารถเข้าถึงได้ตลอดเวลา
- **Body:**
 - (None)

4. Validation

- ไม่มีกระบวนการตรวจสอบสิทธิ์ (Access Control) เพื่อความรวดเร็วในการตรวจสอบสถานะจากระบบ Infrastructure

5. Response

- Success (200 OK / 503 Service Unavailable)
 - คิวค่า 200 OK หากระบบอยู่ในสถานะ healthy หรือ degraded (กรณีส่วนประกอบที่ไม่สำคัญล้มเหลว)
 - คิวค่า 503 Service Unavailable หากระบบอยู่ในสถานะ unhealthy (กรณีส่วนประกอบหลักล้มเหลว)

```
{
  "status": "healthy", // ค่าที่เป็นไปได้: healthy, degraded, unhealthy
  "timestamp": "2026-03-03T14:30:00Z",
  "duration_ms": 123,
  "version": "1.0.0",
  "components": {
    "dynamodb": {
      "status": "healthy",
      "table": "DisasterReports",
      "table_status": "ACTIVE"
    },
    "sqs": {
      "status": "healthy",
      "queue_url": "https://sqs.aws...",
      "approximate_messages": 5
    },
    "gemini": {
      "status": "healthy"
    },
    "eventbridge": {
      "status": "healthy"
    }
  }
}
```

- Error:

- เนื่องจากเป็น API สำหรับตรวจสอบสถานะโดยตรง หากระบบ API Gateway หรือ Lambda ทำงานได้ จะต้องคืนผลลัพธ์ในรูปแบบ JSON ด้านบนเสมอ (แม้สถานะจะเป็น unhealthy ก็ตาม)

6. Dependency

- Internal AWS Dependencies: Amazon DynamoDB, Amazon SQS, Amazon EventBridge
- External AI Dependency: Google Gemini API (gemini-2.5-flash หรือ flash-lite หรือ gemini-2.0-flash)

7. Reliability

- ระบบจะทำการตรวจสอบสถานะเชิงลึก (Deep Health Check) ในแต่ละส่วนประกอบ คือ
 - DynamoDB (Critical): ใช้คำสั่ง DescribeTable เพื่อตรวจสอบว่า Table Status เป็น ACTIVE หรือไม่
 - SQS (Critical): ใช้คำสั่ง GetQueueAttributes เพื่อตรวจสอบการเชื่อมต่อและปริมาณข้อความค้าง
 - Gemini AI (Non-critical): ส่งสัญญาณ Ping หรือตรวจสอบการตอบกลับเบื้องต้นจาก API
 - EventBridge (Non-critical): ใช้คำสั่ง DescribeEventBus เพื่อยืนยันว่า Bus พร้อมรับข้อมูล
- Logic การตัดสินใจสถานะ
 - หาก DynamoDB หรือ SQS ขัดข้อง ระบบจะถือว่าเป็นสถานะ unhealthy (503) เนื่องจากไม่สามารถรับหรือบันทึกข้อมูลได้
 - หาก Gemini หรือ EventBridge ขัดข้อง แต่ส่วนประกอบหลักยังทำงานได้ ระบบจะถือว่าเป็นสถานะ degraded (200) เพื่อให้เจ้าหน้าที่ยังคงเข้าใช้งานระบบได้ตามปกติ

Asynchronous Function Contract

Message Contract #1: Report Verified Event

1. ข้อมูลทั่วไป

- Message Name: ReportVerifiedEvent
- Interaction Style: Asynchronous (Publish/Subscribe)
- Producer: Report Ingestion & Verification Service
- Consumer: Incident Tracking Service
- Channel/Queue: EventBridge Custom Bus ชื่อ disaster-event-bus
- Version: v1

2. คำอธิบาย

- เหตุการณ์นี้จะถูก Publish ออกมาเมื่อเจ้าหน้าที่ทำการยืนยันรายงาน (Verify) และตัดสินใจสร้างเหตุการณ์ใหม่ ข้อมูลที่ผ่านการกรองและสรุปแล้วจะถูกส่งไปยัง Incident Service เพื่อสร้าง Master Record สำหรับการบริหารจัดการภัยพิบัติต่อไป

3. Request

- **Message Headers** (EventBridge Native Fields)
 - Source: service.report-verify (ระบุแหล่งที่มาของเหตุการณ์)
 - Detail-Type: ReportVerifiedEvent (ใช้สำหรับ Rule Filtering ใน EventBridge)
 - EventBusName: disaster-event-bus
 - (หมายเหตุ: ฟิลด์ x-action ไม่ได้ถูกส่งใน Payload เนื่องจาก Logic การตัดสินใจถูกจัดการภายใน API Handler เรียบร้อยแล้ว)
- **Message Body**
 - JSON Data Below

```
{
  "report_ref_id": "r-550e8400-e29b-41d4-a716-446655440000",
  "suggested_incident_data": {
    "type": "FIRE",
    "description": "Fire reported at Central World, smoke visible
from 5km away.",
    "severity_level": 3,
```

```

// [Current Logic] Hardcoded เป็น 3 (ยังไม่ได้คำนวณแบบ Dynamic จาก AI Tags)
"location": {
  "lat": 13.746,
  "lon": 100.539
  //หมายเหตุ: ไม่รวม address_text เนื่องจากระบบยังไม่รองรับ Reverse Geocoding
},
"reporter_count": 1,
// [Current Logic] Hardcoded เป็น 1 (ยังไม่ได้นับจำนวนผู้แจ้งจริงในเวอร์ชันนี้)
"media_evidence": [
  "https://s3.aws.../img1.jpg",
  "https://s3.aws.../img2.jpg"
]
},
"verified_by": "officer_007",
"verification_notes": "Confirmed via traffic camera."
}

```

4. Field Definition

Field	Type	Required?	Description	Validation Rules
report_ref_id	UUID	YES	ID ของ Report ต้นทาง (เพื่อให้ Incident Service เก็บไว้ Trace กลับมาได้)	Must be valid UUID v4 format
suggested_incident_data.type	String	YES	ประเภทเหตุการณ์ที่ระบบ/เจ้าหน้าที่แนะนำ	Enum: FIRE, FLOOD, EARTHQUAKE, ACCIDENT

suggested_incident_data. description	String	YES	รายละเอียดเหตุการณ์ที่ สรุปมาแล้ว	Max 500 chars, No HTML tags
suggested_incident_data. severity_level	Integer	YES	ระดับความรุนแรง (1-5)	1 (Low) - 5 (Critical)
suggested_incident_data. location.lat	Float	YES	พิกัดละติจูด	-90 to 90
suggested_incident_data. location.lon	Float	YES	พิกัดลองจิจูด	-180 to 180
verified_by	String	YES	ID ของเจ้าหน้าที่ผู้ยืนยัน	Non-empty string

5. Validation Rules

- (อยู่ในตารางข้างบน)

6. Response

- **Message Headers**
 - Source: service.incident-tracking
 - Detail-Type: IncidentCreationResultEvent
- **Success Message Body**
 - JSON Data Below

```
{
  "incident_id": "inc-9999-8888",
  "original_report_ref_id": "r-550e8400-e29b-41d4-a716-446655440000",
  "status": "CREATED",
```

```
"timestamp": "2026-03-03T10:30:05Z"

}
```

- Reject/Error Message Body (JSON)

■ JSON Data Below

```
{

"original_report_ref_id": "r-550e8400-e29b-41d4-a716-446655440000",

"status": "FAILED",

"error_code": "DUPLICATE_INCIDENT",

"error_message": "An incident at this location already exists (inc-5555)."

}
```

7. Field Definition

Field	Type	Required	Description	Validation
incident_id	UUID	Yes (Success only)	ID ของ Incident ที่ถูกสร้างสำเร็จ	Valid UUID
original_report_ref_id	UUID	Yes	ID ของ Report ต้นทางที่ส่งไป	Must match original request
status	String	Yes	ผลลัพธ์การทำงาน	Enum: CREATED , FAILED
error_code	String	No	รหัสข้อผิดพลาด (ถ้ามี)	

8. Validation Rules

- (อยู่ในตารางข้างบน)

Message Contract #2: Report Status Change Event

1. ข้อมูลทั่วไป

- Message Name: ReportStatusChangedEvent
- Interaction Style: Asynchronous (Publish/Subscribe / Broadcast)
- Producer: Report Ingestion & Verification Service
- Consumer: Dashboard Service, Notification Service, Property Damage Service
- Channel/Queue: EventBridge Custom Bus ชื่อ disaster-event-bus (แยกชุดข้อมูลด้วย Detail-Type)
- Version: v1

2. คำอธิบาย

- เหตุการณ์นี้จะถูกกระจาย (Broadcast) ออกไปทุกครั้งที่มีการเปลี่ยนแปลงสถานะของ Report เช่น จาก PENDING_REVIEW ไปเป็น SPAM หรือ VERIFIED เพื่อให้ Service อื่น ๆ นำไปอัปเดตหน้าจอ Dashboard แบบ Real-time, ส่งการแจ้งเตือน หรือนำไปคำนวณสถิติเพื่อออกรายงานสรุปผล

3. Request

- **Message Headers** (EventBridge Native Fields)
 - Source: service.report-verify
 - Detail-Type: ReportStatusChangedEvent
 - EventBusName: disaster-event-bus
- **Message Body**
 - JSON Data Below

```
{
  "report_id": "r-1234-5678-9012",
  "old_status": "PENDING_REVIEW",
  "new_status": "SPAM",
  "reason": "Auto-rejected by AI (Trust Score < 30%)",
  "changed_by": "SYSTEM_AI",
```

```
//หมายเหตุ: ใช้ Uppercase "SYSTEM_AI" หรือ ID ของเจ้าหน้าที่ให้เป็นระเบียบเดียวกัน

"timestamp": "2026-03-03T11:15:00Z"

}
```

4. Field Definition

Field	Type	Required	Description	Validation Rules
report_id	UUID	YES	ID ของ Report ที่ถูกเปลี่ยนสถานะ	Must be valid UUID
old_status	String	YES	สถานะเดิมก่อนเปลี่ยน	Enum: RECEIVED, PENDING_REVIEW, VERIFIED, SPAM, DUPLICATE, REJECTED, DELETED
new_status	String	YES	สถานะใหม่ที่ถูกเปลี่ยน	Enum: RECEIVED, PENDING_REVIEW, VERIFIED, SPAM, DUPLICATE, REJECTED, DELETED
reason	String	NO	เหตุผลที่เปลี่ยนสถานะ (สำคัญเวลาโดน Reject)	Max 255 chars
changed_by	String	YES	ผู้เปลี่ยนสถานะ (คน หรือ AI)	ID ของเจ้าหน้าที่ หรือ "SYSTEM_AI"
timestamp	String	YES	เวลาที่เกิดการเปลี่ยนสถานะ	ISO8601 Format

5. Validation Rules

- (อยู่ในตารางข้างบน)

6. Response

- None (This is a Broadcast Event. No response or callback is expected from consumers.)

Service Data

1. Reports Data (Owned by this service)

ตารางหลักที่จัดเก็บรายละเอียดของรายงานภัยพิบัติและผลการวิเคราะห์จาก AI

Field Name	Type	Required	Description	Example
report_id (PK)	UUID	Yes	รหัสอ้างอิงของรายงาน (สร้างโดยระบบเมื่อรับข้อมูล)	r-550e8400-e29b
source_platform	String (Enum)	Yes	แหล่งที่มาของข้อมูล: TWITTER, LINE_OFFICIAL, MOBILE_APP	TWITTER
source_external_id	String	No	ID อ้างอิงจากต้นทาง (เช่น Tweet ID) เพื่อ กันการดิ่งซ้ำ	1758930222345
reporter_id	String	Yes	User ID หรือเบอร์โทร ของผู้แจ้ง (ใช้ระบุ ตัวตน/แบบ)	@somchai_zs
raw_content	Text	Yes	ข้อความดิบที่ได้รับแจ้ง มา	"ไฟไหม้ร้านทอง เยวราช ช่วยด้วย!"
media_urls	List/Array	No	ลิงก์รูปภาพหรือวิดีโอ หลักฐาน	["https://s3.../img1.jpg"]
geo_location	JSON / Object	Yes	พิกัดละติจูด/ลองจิจูด (สำคัญมากสำหรับการ Map)	{ "lat": 13.74, "lon": 100.50 }

ingested_at	DateTime	Yes	เวลาที่ข้อมูลถูกบันทึก ลงระบบ (System Time)	2026-02-18T10:00:00Z
trust_score	Integer (0-100)	Yes	คะแนนความน่าเชื่อถือ ที่ AI ประเมิน (ค่าตั้ง ต้น = 0)	85
ai_analysis_tags	List/Array	No	ป้ายกำกับที่ AI ตรวจจับได้ (ใช้ช่วย Search/Filter)	["FIRE", "URGENT", "SMOKE"]
validation_status	String (Enum)	Yes	สถานะปัจจุบันของ รายงาน: RECEIVED, PENDING_REVIEW, VERIFIED, SPAM, REJECTED, DUPLICATE, DELETED	PENDING_REVIEW
verified_by	String	No	ID ของเจ้าหน้าที่/ ระบบ ที่ทำการเปลี่ยน สถานะล่าสุด	officer_007
verification_notes	Text	No	หมายเหตุจากการ ตรวจสอบ (ทำไมถึง ผ่าน/ไม่ผ่าน)	"ยืนยันจากกล้อง CCTV แล้ว"
linked_incident_id	UUID	No	(Reference) ID ของ Incident จริงที่ รายงานนี้ถูกส่งไปรวม	inc-9988-7766
deleted_by	String	No	ผู้ทำการ Soft Delete (กรณีสถานะ DELETED)	admin_001

deleted_reason	String	No	เหตุผลในการลบข้อมูล	"Duplicate/PII"
potential_duplicates	List[UUID]	No	รายการ ID รายงานที่ระบบประเมินว่าอาจซ้ำซ้อน	รายการ ID รายงานที่ระบบประเมินว่าอาจซ้ำซ้อน
updated_at	DateTime	Yes	เวลาที่อัปเดตข้อมูลล่าสุด	2026-03-03T10:10:00Z
ai_analysis_failed	Boolean	No	Flag ระบุหากกระบวนการ AI ขัดข้อง	false
ai_reasoning	String	No	เหตุผลจาก AI ที่อธิบายที่มาของคะแนน	"Detected smoke in image..."
suggested_category	String (Enum)	No	ประเภทเหตุการณ์ที่ AI แนะนำ	FIRE

2. Report Audit Logs (Owned by this service)

ตารางจัดเก็บประวัติการเปลี่ยนแปลงข้อมูล เพื่อใช้ในการตรวจสอบย้อนหลัง

Field Name	Type	Required	Description	Example
log_id (PK)	UUID	Yes	รหัสประจำรายการ Log	log-1122-3344
report_ref_id	UUID	Yes	(FK) อ้างอิงไปยัง Report ตัวไหน	r-550e8400-e29b
actor_id	String	Yes	ID ของผู้กระทำ (User หรือ SYSTEM_AI)	officer_007
action_type	String	Yes	ประเภทการกระทำ: STATUS_CHANGE,	STATUS_CHANGE

			DATA_EDIT, SOFT_DELETE, AI_ANALYSIS	
previous_value	JSON/String	No	ค่าเดิมก่อนแก้ไข (เพื่อดูความเปลี่ยนแปลง)	{ "status": "PENDING" }
new_value	JSON/String	Yes	ค่าใหม่หลังแก้ไข	{ "status": "VERIFIED" }
timestamp	DateTime	Yes	เวลาที่เกิดการกระทำ	2026-02- 18T10:05:00Z

Technical Note: ระบบมีการสร้าง GSI: gsi_report_timestamp (Partition Key: report_ref_id, Sort Key: timestamp) เพื่อรองรับการ Query ประวัติการเปลี่ยนแปลงรายรายงานโดยเรียงตามลำดับเวลาได้อย่างรวดเร็ว

3. StatsCounter Table (Atomic Counter)

ตารางสำหรับเก็บสถิติแบบสรุปยอด (Aggregated Stats) เพื่อสนับสนุน Dashboard โดยไม่ต้อง Scan ตารางหลัก

Field Name	Type	Required	Description	Example
stat_key (PK)	String	Yes	รูปแบบ: วัน#ชื่อ counter	2026-03- 03#total_received
stat_value	Number	Yes	ค่านับสะสม (อัปเดตด้วย Atomic Increment)	1500

4. ข้อมูลเพิ่มเติม

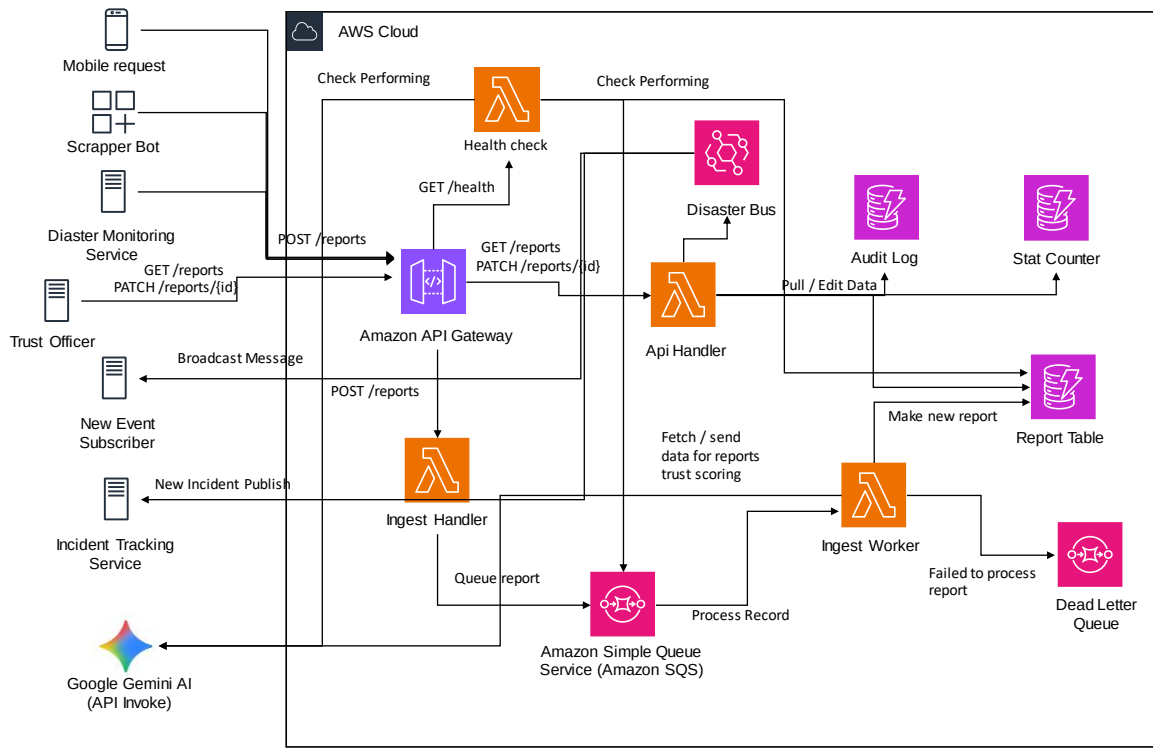
- Idempotency Strategy: การใช้ source_external_id เป็นหัวใจสำคัญในการรับข้อมูลจาก Social Media หากระบบภายนอกส่งข้อมูลเดิมเข้ามาหลายครั้ง เราจะใช้ฟิลต์นี้ตรวจสอบก่อนออก report_id ใหม่ เพื่อลดความซ้ำซ้อนตั้งแต่ต้นทาง
- Audit Logging Strategy: ทุกการเปลี่ยนแปลงสถานะหรือเนื้อหา จะต้องถูกบันทึกลง Audit Logs เสมอ โดยระบุ action_type ให้ชัดเจน เช่น หากลบข้อมูลจะบันทึกเป็น SOFT_DELETE

และหากเป็นการประมวลผลจาก Gemini จะบันทึกเป็น AI_ANALYSIS เพื่อแยกแยะระหว่างการทำงานของมนุษย์และปัญญาประดิษฐ์

- Performance Optimization: ตาราง StatsCounter ถูกแยกออกมาเพื่อแก้ปัญหา Performance ในการทำ Dashboard โดยเฉพาะ ทำให้เราสามารถแสดงผล "ยอดรวมผู้แจ้งเหตุวันนี้" ได้ภายในเวลาไม่กี่มิลลิวินาที

Service Architecture

1. ภาพแสดงการทำงานของแต่ละ Function



2. Components

- User / Client
 - External Sources: Social Media Scrapers, Mobile Apps ที่ส่งข้อมูลเข้ามา
 - Trust Officer: เจ้าหน้าที่ผู้ใช้งานผ่านหน้า Web Dashboard เพื่อตรวจสอบข้อมูล
- Amazon API Gateway
 - ทำหน้าที่เป็นประตูหลัก (Entry Point) รับคำขอทั้งแบบ Synchronous (จากเจ้าหน้าที่) และ Asynchronous Ingestion (จาก Bot)
- Amazon SQS (report-ingestion-queue)
 - คิวสำหรับพักข้อมูล (Buffer) จาก External Sources เพื่อรองรับ Traffic มหาศาล (Spike Traffic) ป้องกันระบบล่ม และช่วยลด Rate Limit
- AWS Lambda (Ingestion Worker)
 - ฟังก์ชันเบื้องหลังที่ตื่นขึ้นมาเมื่อมีข้อมูลใน SQS ทำหน้าที่เรียก AI มาประมวลผล (Scoring), ตรวจสอบข้อมูลซ้ำ (Deduplicate), และบันทึกลงฐานข้อมูล

- Amazon DynamoDB (Reports Table)
 - ฐานข้อมูล NoSQL หลักของ Service เก็บข้อมูล Reports ทั้งหมด, สถานะการตรวจสอบ, และ Audit Logs รองรับการอ่าน/เขียนความเร็วสูง
- AWS Lambda (API Handler)
 - ฟังก์ชันสำหรับให้บริการหน้า Dashboard (Get List, View Detail) และประมวลผลคำสั่งยืนยัน (Verify) จากเจ้าหน้าที่
- Amazon EventBridge (Custom Event Bus)
 - ช่องทางสื่อสารหลักไปยัง Service อื่นๆ (Event Bus) เมื่อมีการยืนยันข้อมูล (Verified) ระบบจะส่งข้อมูลบอก EventBridge เพื่อให้ Service ปลายทาง (เช่น Incident Service) มารับข้อมูลไปทำงานต่อ
- Google Gemini API (External AI Service)
 - บริการ AI อัจฉริยะที่ทำหน้าที่วิเคราะห์เนื้อหาของรายงานดิบทั้งข้อความและรูปภาพ เพื่อประเมินระดับความน่าเชื่อถือ สกัดคีย์เวิร์ดสำคัญ เช่น ประเภทยั่วยุ ระเบิด ความรุนแรง และช่วยคัดกรองข่าวปลอมหรือสแปมโดยอัตโนมัติ ก่อนส่งผลลัพธ์ให้ Ingestion Worker บันทึกลงฐานข้อมูลเพื่อลดภาระการคัดกรองของเจ้าหน้าที่

3. Explanation เพิ่มเติม

- สถาปัตยกรรมของ Report Ingestion & Verification Service ถูกออกแบบโดยเน้นความทนทาน (Resilience) และการแยกส่วนการทำงาน (Decoupling) โดยแบ่งออกเป็น 3 ส่วนหลัก ดังนี้ คือ

1. ส่วนการรับข้อมูลและประมวลผลเบื้องต้น (Asynchronous Ingestion)

เนื่องจากการแจ้งเหตุภัยพิบัติมักมีปริมาณมหาศาลและมาพร้อมกันในเวลาสั้นๆ ระบบจึงใช้ Amazon SQS เข้ามาคั่นกลางระหว่าง API Gateway และส่วนประมวลผล เมื่อ External Sources ส่งข้อมูลเข้ามา API Gateway จะส่งข้อมูลลง SQS และตอบกลับทันที (202 Accepted) เพื่อลดระยะเวลารอคอย จากนั้น AWS Lambda (Ingestion Worker) จะดึงข้อมูลจาก SQS ไปประมวลผลทีละ Batch โดยมีการเรียกใช้ Google Gemini API เพื่อวิเคราะห์ความน่าเชื่อถือ และตรวจสอบความซ้ำซ้อนกับข้อมูลใน Amazon DynamoDB ก่อนบันทึก กระบวนการนี้ช่วยให้ระบบรองรับ Load ได้ไม่จำกัดโดยไม่กระทบต่อฐานข้อมูลหลัก

2. ส่วนการตรวจสอบและบริหารจัดการ (Synchronous Management)

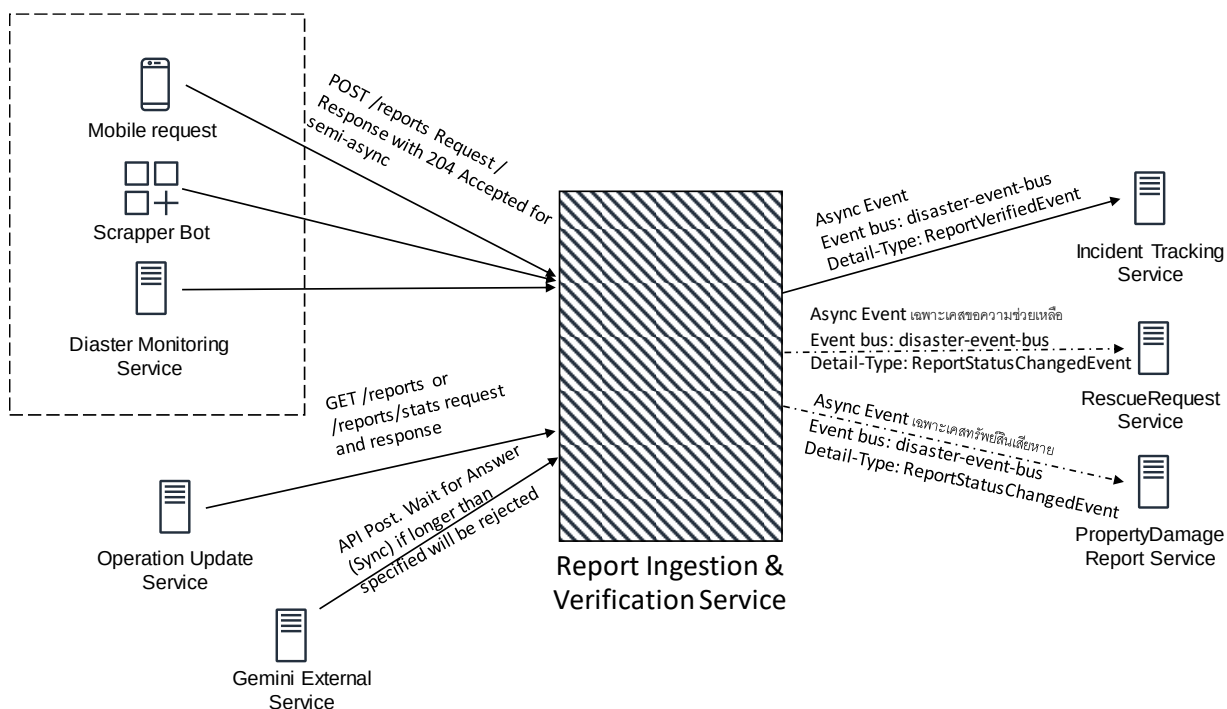
สำหรับเจ้าหน้าที่ Trust Officer ที่ต้องคัดกรองข้อมูล ระบบใช้รูปแบบ REST API ผ่าน API Gateway เชื่อมต่อกับ AWS Lambda (API Handler) โดยตรง เพื่อดึงรายการที่ AI คัดกรองแล้ว (Pending Review) มาแสดงผลและรองรับคำสั่งตรวจสอบยืนยัน (Verify) หรือปฏิเสธ (Reject) โดยมีการอ่านและอัปเดตสถานะลงใน DynamoDB แบบ Real-time เพื่อให้เจ้าหน้าที่เห็นสถานะล่าสุดทันที

3. ส่วนการส่งต่อข้อมูล (Event-Driven Broadcast)

เมื่อเจ้าหน้าที่ทำการยืนยันข้อมูล (Status = VERIFIED) ระบบจะไม่เรียก API ของ Service อื่นโดยตรง (เพื่อป้องกันการผูกติดกันเกินไป) แต่ AWS Lambda จะทำการ Publish Event ชื่อ ReportVerified ไปยัง Amazon EventBridge ซึ่งทำหน้าที่เป็น Router กลาง เพื่อกระจายข่าวดังกล่าวไปยัง Service ที่เกี่ยวข้อง เช่น Incident Tracking Service ให้นำข้อมูลไปสร้าง Incident ต่อไป รูปแบบนี้ช่วยให้ระบบมีความเป็นอิสระ (Autonomy) และง่ายต่อการขยาย Service ใหม่ๆ ในอนาคต

Service Interaction

1. ภาพ Diagram



2. Upstream Services

เป็นบริการต้นทางที่เรียกใช้งาน Report Ingestion & Verification Service

- DisasterMonitoring Service (IoT & Sensors)
 - Interaction Type: Synchronous (HTTP POST)
 - Action: เรียกใช้งาน API POST /reports
 - Description: เมื่อเซ็นเซอร์ตรวจจับความผิดปกติทางกายภาพได้ เช่น ระดับน้ำสูงเกิน พิกัดเตือนภัย บริการนี้จะทำการส่งข้อมูลดิบเข้าสู่ระบบในรูปแบบ Report เพื่อให้ Report Ingestion & Verification Service นำไปประมวลผลและเทียบเคียงกับข้อมูลจากสื่อสังคมออนไลน์ ซึ่งจะช่วยเพิ่มความแม่นยำในการยืนยันเหตุการณ์จริง
- Operation Update Service (Command Center Dashboard)
 - Interaction Type: Synchronous (HTTP GET)
 - Action: เรียกใช้งาน API GET /reports/stats หรือ GET /reports?status=PENDING
 - Description: ทำหน้าที่ดึงข้อมูลเชิงสถิติ เช่น ยอดรวมการรายงานเหตุ (Total Reports), พื้นที่ที่มีความหนาแน่นของการแจ้งเหตุแบบ Heatmap Data และรายการที่รอการตรวจสอบ เพื่อนำไปแสดงผลบนระบบจัดการส่วนกลางแบบ Dashboard

สำหรับสนับสนุนการตัดสินใจของผู้บัญชาการ และสร้างความตื่นตัวต่อสถานการณ์
ภาพรวม (Situational Awareness)

3. Downstream Services

เป็นบริการปลายทางที่ Report Ingestion & Verification Service เรียกหรือส่งข้อมูลไป

- IncidentTracking Service (Consumer หลัก)
 - Interaction Type: Asynchronous (Event-Driven ผ่าน EventBridge/PubSub)
 - Event Name: ReportVerified (Topic: report.verified)
 - Description: บริการนี้เป็นกลไกหลักในการรับช่วงต่อ เมื่อรายงานได้รับการตรวจสอบและยืนยันความถูกต้อง (Status = VERIFIED) ระบบจะทำการกระจายเหตุการณ์ Publish Event ออกไป เพื่อให้ IncidentTracking Service รับข้อมูลที่ได้ Subscribe และนำไปประมวลผลเพื่อสร้างเหตุการณ์ใหม่ หรือผสานข้อมูลเข้ากับเหตุการณ์เดิมโดยอัตโนมัติ กระบวนการนี้ช่วยให้ข้อมูลไหลเวียนได้อย่างต่อเนื่องโดยไม่ต้องรอการตอบกลับจาก API ของระบบปลายทาง
- RescueRequest Service (Consumer รอง - เฉพาะเหตุ SOS)
 - Interaction Type: Asynchronous (Event-Driven)
 - Event Name: ReportVerified (Filter: type=SOS or severity=CRITICAL)
 - Description: หากรายงานที่ผ่านการยืนยันแล้ว มีเนื้อหาเกี่ยวข้องกับการขอความช่วยเหลือเร่งด่วน เช่น ผู้ประสบภัยติดค้าง ผู้ป่วยวิกฤต บริการนี้จะคัดกรองและนำข้อมูลดังกล่าวไปสร้างเป็น คำสั่งปฏิบัติการกู้ภัยในทันที เพื่อลดความซ้ำซ้อนและเร่งระยะเวลาในการบันทึกข้อมูลของเจ้าหน้าที่กู้ภัย
- PropertyDamageReport Service (Consumer รอง - เฉพาะเหตุความเสียหาย)
 - Interaction Type: Asynchronous (Event-Driven)
 - Event Name: ReportVerified (Filter: type=DAMAGE)
 - Description: หากรายงานเกี่ยวข้องกับความเสียหายทางกายภาพหรือโครงสร้างพื้นฐาน เช่น ถนนขาด สะพานพัง อาคารทรุดตัว โดยไม่มีผู้ได้รับบาดเจ็บ ข้อมูลจะถูกส่งต่อไปยังบริการนี้ เพื่อนำไปบันทึกเป็นฐานข้อมูลรายการความเสียหายที่รอการซ่อมแซมหรือประเมินมูลค่าทางวิศวกรรมต่อไป
- Incident Service (Reference Check - Optional but Good)
 - Interaction Type: Synchronous (HTTP GET)

- Action: เรียกใช้งาน API GET /incidents?active=true
- Description: ในขั้นตอนที่เจ้าหน้าที่กำลังพิจารณายืนยันความถูกต้อง และต้องการเชื่อมโยงรายงานเข้ากับเหตุการณ์ที่กำลังเกิดขึ้น ระบบส่วนหน้า (Dashboard) จะทำการเรียกใช้งาน API นี้เพื่อดึงรายชื่อเหตุการณ์ที่ยังเปิดอยู่มาแสดงผลให้เจ้าหน้าที่เลือก เพื่อเป็นการรักษาความถูกต้องและสอดคล้องของข้อมูลในระบบ

4. เหตุผลที่ออกแบบแบบนี้ (Technical Rationale)

- การเลือกใช้สถาปัตยกรรมแบบ Asynchronous Event-Driven โดยให้ระบบส่งเหตุการณ์แจ้งเตือนความสำเร็จ (ReportVerified) ออกไปยังหลายบริการปลายทาง ได้แก่ IncidentTracking RescueRequest PropertyDamage พร้อม ๆ กันนั้น เป็นการประยุกต์ใช้รูปแบบการกระจายข้อมูลแบบ Fan-out Pattern ซึ่งมีประสิทธิภาพสูงในการจัดการข้อมูลจำนวนมาก
- ข้อได้เปรียบของการออกแบบในลักษณะนี้คือการที่ Report Ingestion & Verification Service ไม่จำเป็นต้องรับรู้ถึงตรรกะทางธุรกิจของบริการอื่น มีหน้าที่เพียงประมวลผลและประกาศว่าข้อมูลนี้ได้รับการยืนยันแล้ว ส่วนบริการปลายทางสามารถพิจารณาและดึงข้อมูลไปดำเนินการต่อได้ตามความรับผิดชอบของตนเอง
- สถาปัตยกรรมนี้ช่วยเพิ่มความทนทานต่อข้อผิดพลาดให้กับระบบภาพรวม กล่าวคือ หากมีบริการปลายทางใดบริการหนึ่งเกิดปัญหาขัดข้อง เช่น ระบบ RescueRequest ล่มชั่วคราว ระบบ Report Ingestion ซึ่งเป็นระบบหลัก จะยังคงสามารถรับแจ้งและประมวลผลข้อมูลต่อไปได้ตามปกติ โดยไม่เกิดสภาวะล้มเหลวแบบลูกโซ่ (Cascading Failure)

Dependency Mapping

Report Ingestion & Verification Service

1. Incident Tracking Service

- Type: Microservice (External)
- Interaction Style: Synchronous (REST API via HTTP GET)
- Purpose: เรียกดูรายการเหตุการณ์ที่กำลังดำเนินอยู่ (Active Incidents) เพื่อให้เจ้าหน้าที่ (Trust Officer) สามารถเลือกเชื่อมโยงรายงาน (Link Report) เข้ากับเหตุการณ์ที่มีอยู่แล้วได้ถูกต้อง แทนที่จะสร้างเหตุการณ์ใหม่ซ้ำซ้อน
- Criticality: Medium (Service ยังทำงานต่อได้ แต่ฟีเจอร์การ Link จะใช้งานไม่ได้)
- Failure Handling:
 - Graceful Degradation: หาก Incident Tracking Service ไม่ตอบสนอง ระบบจะปิดการใช้งานปุ่ม "Link to Existing Incident" ชั่วคราว (Disable UI Option)
 - Fallback: บังคับให้เจ้าหน้าที่เลือก "Create New Incident" หรือบันทึกสถานะเป็น VERIFIED ไว้ก่อน แล้วค่อยมาทำการ Merge ข้อมูลในภายหลังเมื่อระบบกลับมาปกติ

2. Amazon SQS (report.ingestion.queue.v1)

- Type: Queue (AWS Service)
- Interaction Style: Asynchronous (Buffer & Load Leveling)
- Purpose: เป็นจุดพักข้อมูล (Buffer) สำหรับรับ Raw Reports จำนวนมหาศาลจาก External Sources (Bots, Apps) เพื่อป้องกันไม่ให้ Database หรือระบบประมวลผลล่มในช่วงที่มี Traffic Spike (เช่น ช่วงเกิดภัยพิบัติรุนแรง)
- Criticality: Critical (ถ้า SQS ล่ม ข้อมูลขาเข้าจะสูญหาย)
- Failure Handling:
 - ใช้ Retry Policy ของ AWS Lambda ในการพยายามดึงข้อมูลไปประมวลผลซ้ำหากเกิด Error ชั่วคราว
 - หากประมวลผลล้มเหลวครบตามจำนวนที่กำหนด (Max Retries Exceeded) ข้อมูลจะถูกย้ายไปยัง Dead Letter Queue (DLQ) โดยอัตโนมัติ เพื่อให้ทีม Developer เข้ามาตรวจสอบและ Re-drive ข้อมูลกลับเข้าระบบในภายหลัง (Zero Data Loss)

3. Amazon EventBridge (disaster.event.bus.v1)

- Type: Event Bus (AWS Service)
- Interaction Style: Asynchronous (Event Publishing / Fan-out)
- Purpose: เป็นช่องทางหลักในการกระจายข่าวเมื่อรายงานได้รับการยืนยัน (Event: ReportVerified) ไปยัง Service ปลายทางหลายตัวพร้อมกัน (เช่น Incident Service, Rescue Service, Dashboard) โดยไม่ต้องเชื่อมต่อกันโดยตรง
- Criticality: High (ถ้าส่ง Event ไม่ได้ Service อื่นจะไม่รู้ว่ามีเหตุเกิด)
- Failure Handling:
 - หาก Publish Event ไม่สำเร็จ ระบบจะบันทึก Error Log พร้อม Payload ลงใน Database (Local Outbox Pattern)
 - มีระบบ Scheduled Retry Task ที่จะคอยกวาด Event ที่ส่งไม่ผ่านใน Database เพื่อทำการส่งใหม่อีกครั้งจนกว่าจะสำเร็จ

4. Amazon DynamoDB (Reports Table)

- Type: Database (NoSQL)
- Interaction Style: Internal Data Access (Read/Write)
- Purpose: เป็นแหล่งข้อมูลหลัก (Single Source of Truth) ของ Service จัดเก็บข้อมูล Raw Reports ทั้งหมด, ผลการวิเคราะห์จาก AI, สถานะการตรวจสอบ (Validation Status), และ ประวัติการแก้ไข (Audit Logs)
- Criticality: Critical (ระบบจะหยุดทำงานทันทีหาก Database ล่ม)
- Failure Handling:
 - Write Failure: หากบันทึกข้อมูลไม่สำเร็จ API จะตอบกลับ Error 500 (กรณี Sync) หรือโยน Exception เพื่อให้ SQS Retry (กรณี Async)
 - Throttling: หากมีการเขียนเกิน Read/Write Capacity Unit ระบบจะใช้ Exponential Backoff ในการรอและลองเขียนใหม่

5. Google Gemini API

- Type: AI Service (External API)
- Interaction Style: Synchronous (Request/Response ผ่าน REST/RPC)
- Purpose: วิเคราะห์ข้อความในรายงานเพื่อคำนวณค่าความน่าเชื่อถือ (Trust Score), สกัด Keywords, และคัดกรอง Spam

- Criticality: Medium (ระบบยังทำงานได้แม้ API ล่ม)
- Failure Handling:
 - Fallback to Neutral: หาก API ไม่ตอบสนอง (Timeout) หรือติด Rate Limit ระบบจะตั้งค่า Trust Score เป็น 50 (Unknown)
 - Flag for Review: ติดป้ายกำกับว่า AI Analysis Failed และส่งเข้าคิว Manual Review เพื่อให้มนุษย์ตรวจสอบ 100%